420-635-AB – NETWORK INSTALLATION AND ADMINISTRATION I

PROJECT 2
Report: Apache Virtual Hosts and Access Control Configuration

PREPARED BY: GUILLERMO PADILLA KEYMOLE

John Abbott College | Network Administration Program

Contents

Table of Figures

Introduction

This project focuses on configuring advanced Apache virtual hosting environments on an AlmaLinux server. It builds on the foundational knowledge from Project Part I and introduces more complex scenarios, including name-based hosting, port-based hosting, IP-specific virtual servers, and dynamic virtual hosting using Apache's VirtualDocumentRoot.

Each task in this project explores a different method of hosting multiple websites on a single Apache server, using various combinations of domain names, ports, and IP addresses. The project also introduces access control rules based on client subnets, simulating real-world web hosting and security requirements.

All configurations were tested using an Ubuntu client to ensure proper hostname resolution, access control enforcement, and functional website delivery. The default project root for this lab is /var/www/html_project2, with additional virtual directories placed under /var/www/virtuals.

This report documents every step taken to configure, validate, and troubleshoot the virtual host environments. It includes terminal commands, screenshots, and a master index page to test each scenario through hyperlinks. By completing this project, I gained hands-on experience with Apache virtual hosting, hostname resolution, log file separation, and network-based web access control, all of which are essential in real-world server administration.

Project II Diagram

This diagram shows how our Apache web server is organized for Project Part II. It maps out how the Ubuntu client connects to different AlmaLinux IP addresses and how each task points to its specific web directories, all managed by virtual hosts.

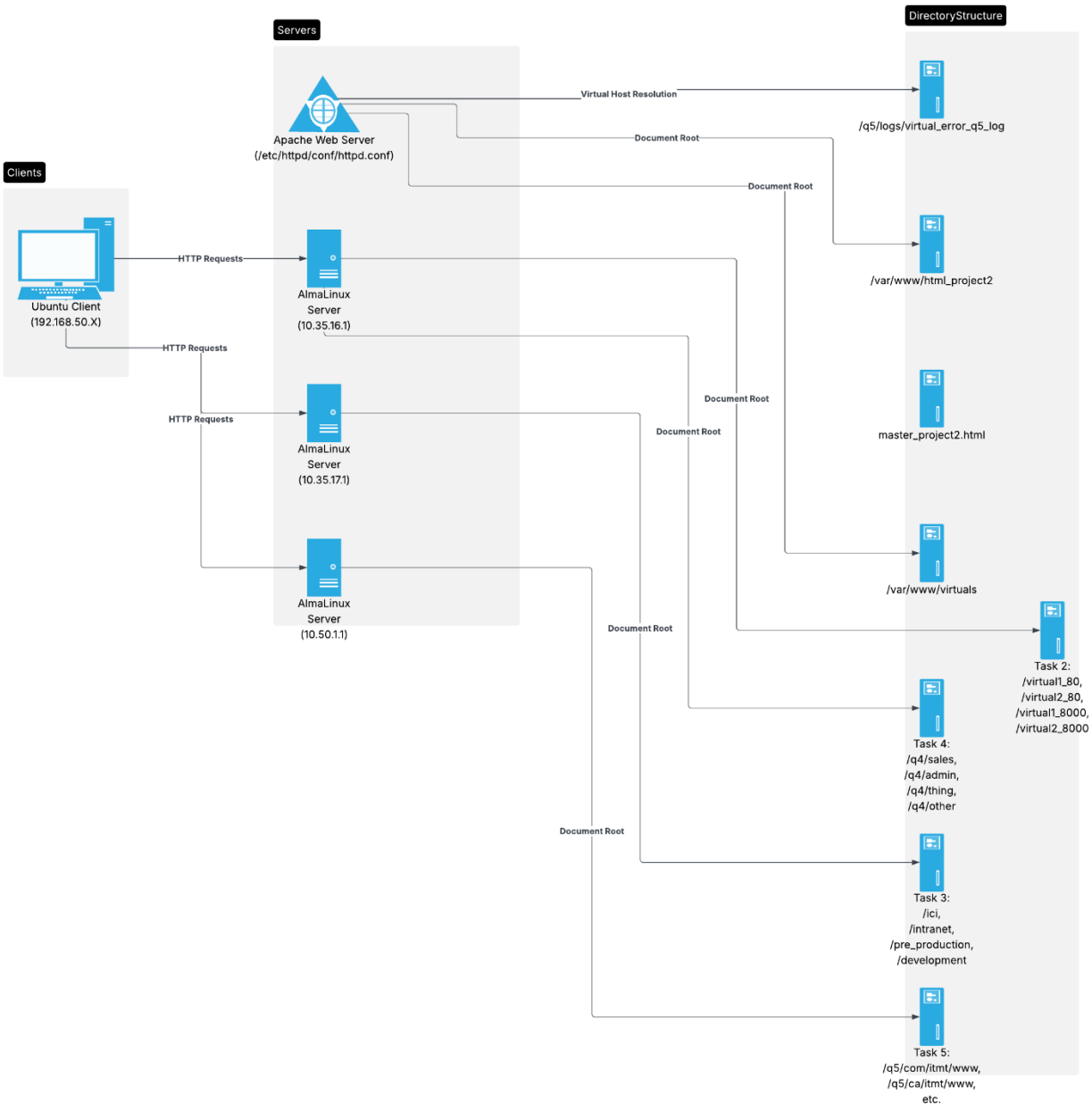

Figure 1 Apache Web Server – Virtual Hosts and Directory Structure Overview

TASK 1 – PREPARATION

This first task lays the foundation for the rest of the Apache Virtual Host project. The goal is to prepare a dedicated web root directory for the project, configure Apache to serve content from this new location, apply the correct SELinux policies and permissions, configure the firewall, and finally, build a simple homepage that links to each task. These steps simulate how real web servers are structured and secured in professional environments.

Create the project root directory

I began by creating a new directory at /var/www/html_project2 using the mkdir command. This directory will act as the root for all the content I will create and serve with Apache. In professional setups, web servers usually serve files from dedicated root folders, and in this case, I'm customizing it for this project. The -p option is helpful because it creates any necessary parent directories if they don't exist already.

```
root@server07 ~ $ mkdir -p /var/www/html_project2
```

Figure 2 Creating the project root directory /var/www/html_project2.

Set Correct Permissions

Next, I changed the permissions of the project folder to allow Apache to read and access its contents. The command below gives the owner full access and everyone else read and execute rights, which is standard for web-accessible folders. This step ensures that the web server can load and display any HTML pages or other content placed in this folder.

```
root@server07 ~ $ chmod -R 755 /var/www/html_project2
root@server07 ~ $
```

Figure 3 Setting read/execute permissions with chmod for the Apache-accessible directory.

Apply SELinux Context for Apache

On systems like AlmaLinux where **SELinux** is enabled, we must also give Apache special permissions at the security policy level. SELinux doesn't rely on basic permissions like chmod, it uses contexts to determine which services can access which files. This command sets the correct SELinux context so Apache can serve files from the directory.

```
root@server07 ~ $ chcon -R -t httpd_sys_rw_content_t /var/www/html_project2
root@server07 ~ $
```

Figure 4 Applying the httpd_sys_rw_content_t context to allow Apache access.

To simplify how Apache accesses different content areas, I also enabled a SELinux boolean called httpd_unified. This tells SELinux to allow Apache to treat content uniformly (instead of checking overly strict rules for each folder).

```
root@server07 ~ $ setsebool -P httpd_unified 1
root@server07 ~ $
```

Figure 5 Enabling the SELinux unified write access policy for Apache.

Create the homepage (master_project2.html)

To test all my virtual hosts later in the project, I created a homepage that acts like a dashboard. It contains hyperlinks to each future server configuration I will create in Task 2.

```
root@server07 ~ $ vim /var/www/html_project2/master_project2.html
```

Here is the HTML I used:

```
<!DOCTYPE html>
<html>
<head>
    <title>Apache Virtual Hosts</title>
</head>
<body>
    <h1>Project Part 2</h1>
    <p>************************************</p>
    <h2>Task 2</h2>

    <a href="http://virtual1.aucegep.com/">virtual1.aucegep.com:80</a><br>
    <a href="http://virtual1.aucegep.com:8000/">virtual1.aucegep.com:8000</a><br>
    <a href="http://virtual2.aucegep.com/">virtual2.aucegep.com:80</a><br>
    <a href="http://virtual2.aucegep.com:8000/">virtual2.aucegep.com:8000</a><br>
</body>
</html>
```

Figure 6 HTML homepage created to provide testing links for each virtual host task.

Update Apache to Use the New Directory

Apache is preconfigured to serve web files from /var/www/html. Since I'm using /var/www/html_project2, I updated Apache's configuration to point to the correct root.

First, I made a backup of the configuration file:

```
root@server07 ~ $ cp /etc/httpd/conf/httpd.conf /etc/httpd/conf/httpd.conf.bak
root@server07 ~ $
```

Then I opened the config file and updated the DocumentRoot directive:

```
root@server07 ~ $ vim /etc/httpd/conf/httpd.conf
```

```
DocumentRoot "/var/www/html_project2"
```

Figure 7 DocumentRoot updated to point to the project directory.

Also, I updated the <Directory> section to match the new path:

```
<Directory "/var/www/html_project2">
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Figure 8 Matching <Directory> path modified to grant proper access to Apache.

Set the Default Page (DirectoryIndex)

Normally, Apache looks for a file named index.html when a user visits a site. But I named my homepage master_project2.html, so I had to tell Apache to load that file by default instead.

Inside httpd.conf, I modified (This ensures that going to 192.168.50.10 shows the right page):

```
<IfModule dir_module>
    DirectoryIndex master_project2.html
</IfModule>
```

Figure 9 Setting master_project2.html as the default landing page for Apache.

Allow Apache Through the Firewall

Web servers must have their ports open in the firewall so other computers can connect. I used firewall-cmd to open the ports needed for this project.

```
root@server07 ~ $ firewall-cmd --permanent --add-service=http --zone=nm-shared
success
root@server07 ~ $
```

```
root@server07 ~ $ firewall-cmd --permanent --add-port=8000/tcp --zone=nm-shared
success
root@server07 ~ $ firewall-cmd --permanent --add-port=8080/tcp --zone=nm-shared
success
root@server07 ~ $ firewall-cmd --permanent --add-port=8081/tcp --zone=nm-shared
success
root@server07 ~ $ firewall-cmd --permanent --add-port=8082/tcp --zone=nm-shared
success
root@server07 ~ $ firewall-cmd --permanent --add-port=8083/tcp --zone=nm-shared
success
root@server07 ~ $
```

```
root@server07 ~ $ firewall-cmd --reload
success
root@server07 ~ $
```

Figure 10 Firewall configuration to allow Apache and required ports for later virtual hosts.

Allow Custom Ports in SELinux

Even if the firewall allows access, SELinux will still block Apache from using ports like 8000 or 8080 unless we tell it otherwise. I used semanage to register these extra ports.

```
root@server07 ~ $ semanage port -a -t http_port_t -p tcp 8000
Port tcp/8000 already defined, modifying instead
root@server07 ~ $ semanage port -a -t http_port_t -p tcp 8080-8083
root@server07 ~ $ semanage port -l | grep http_port_t
http_port_t                    tcp      8080-8083, 8000, 80, 81, 443, 488, 8008, 8009, 8443, 9000
pegasus_http_port_t            tcp      5988
root@server07 ~ $
```

Figure 11 SELinux updated to authorize non-standard ports for Apache.

Enable and Start Apache

I enabled the Apache service so it starts automatically with the system, and I started it right away:

```
root@server07 ~ $ systemctl enable --now httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/s
ervice.
root@server07 ~ $ systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: active (running) since Mon 2025-04-21 10:16:59 EDT; 20s ago
       Docs: man:httpd.service(8)
   Main PID: 4564 (httpd)
```

Test Apache Configuration Syntax

Before restarting the service, I tested the Apache configuration to make sure there were no errors. This is a good habit to avoid issues from typos or misconfigurations.

```
root@server07 ~ $ httpd -t
Syntax OK
root@server07 ~ $
```

Figure 12 httpd -t confirms configuration is valid and ready to use.

Reload Apache

To apply the changes, I reloaded the Apache service:

```
root@server07 ~ $ systemctl reload httpd
root@server07 ~ $
```

Verify in Browser

Finally, I opened my browser and went to: 192.168.50.10. The master_project2.html homepage loaded successfully and showed the Task 2 test links.

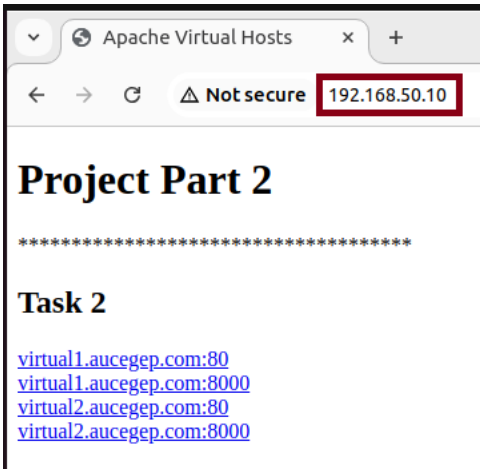

Figure 13 Homepage loaded successfully in browser, showing working links for Task 2.

TASK 2 – Name and Port-Based Virtual Hosts

In this task, I configured multiple virtual websites hosted on the same IP address (10.35.16.1) but using different TCP ports and hostnames. Each website is associated with a unique directory and serves different content. This simulates real-world hosting where multiple web applications share the same server but respond on different ports or domain names.

We configured the following combinations:

virtual1.aucegep.com:80

virtual1.aucegep.com:8000

virtual2.aucegep.com:80

virtual2.aucegep.com:8000

Each pairing serves content from its own directory.

Create Directories for Each Website

I created four distinct directories under /var/www/virtuals, one for each virtual host:

- /var/www/virtuals/virtual1_80
- /var/www/virtuals/virtual1_8000
- /var/www/virtuals/virtual2_80
- /var/www/virtuals/virtual2_8000

```
root@server07 ~ $ mkdir -p /var/www/virtuals/virtual1_80 /var/www/virtuals/virtual1_8000 /var/www/virtuals/
virtual2_80 /var/www/virtuals/virtual2_8000
root@server07 ~ $
```

Figure 14 Creating the project folders for each port-based virtual host.

Create HTML Pages and Set Proper Permissions

After creating the project directories, I needed to place content inside them and ensure Apache had permission to serve those files.

Each of the four websites has its own folder, so I created a simple index.html file in each directory with a unique welcome message to help me identify which virtual host was working during testing

```
root@server07 ~ $ echo "<h1>Welcome to virtual1.aucegep.com on port 80</h1>" > /var/www/virtuals/virtual1_80/index.html
root@server07 ~ $ echo "<h1>Welcome to virtual1.aucegep.com on port 8000</h1>" > /var/www/virtuals/virtual1_8000/index.html
root@server07 ~ $ echo "<h1>Welcome to virtual2.aucegep.com on port 80</h1>" > /var/www/virtuals/virtual2_80/index.html
root@server07 ~ $ echo "<h1>Welcome to virtual2.aucegep.com on port 8000</h1>" > /var/www/virtuals/virtual2_8000/index.html
root@server07 ~ $
```

Figure 15 Creating a separate homepage for each virtual site and port.

Set directory permissions

Apache must be able to **read and execute** the content inside these directories. So, I gave the correct permissions using chmod.

This sets the directories to:

- Owner: read, write, execute
- Group & Others: read and execute (so Apache can access the content)

```
root@server07 ~ $ chmod -R 755 /var/www/virtuals/
root@server07 ~ $
```

Figure 16 Setting folder permissions with chmod 755 for Apache access.

Apply SELinux context to allow Apache access

Because SELinux is enforced in AlmaLinux, regular file permissions are **not enough**. Apache needs a specific SELinux **context** to access and serve files from custom directories.

I applied the following command to recursively set the correct context:

```
root@server07 ~ $ chcon -R -t httpd_sys_content_t /var/www/virtuals/
root@server07 ~ $
```

Figure 17 Applying httpd_sys_content_t SELinux context for Apache read access.

This context (httpd_sys_content_t) allows Apache to read files in the directory.

If I later want to allow **write access** (e.g., for uploads), I could use httpd_sys_rw_content_t. But for serving static HTML pages, read-only access is enough.

Configure Local Name Resolution with /etc/hosts

In a real-world production environment, websites use public **DNS servers** to translate domain names like virtual1.aucegep.com into IP addresses. However, since we're working in a **lab environment** without DNS, we simulate name resolution using the local /etc/hosts file on both the **AlmaLinux server** and the **Ubuntu client**.

This ensures that when we type the domain name in the browser or use curl, the system knows it should connect to our Apache server at 192.168.50.10.

What is /etc/hosts?

The /etc/hosts file is a local static table used to map domain names to IP addresses **without using DNS**. Any mappings we add here will take priority when resolving names, making it perfect for lab testing.

On both the **server (AlmaLinux)** and the **client (Ubuntu)**, I opened the /etc/hosts file with a text editor:

```
root@server07 ~ $ vim /etc/hosts
```

```
gkeymole@client07:~$ sudo vim /etc/hosts
[sudo] password for gkeymole:
gkeymole@client07:~$
```

Then I added the following two lines at the end of the file:

```
10.35.16.1  virtual1.aucegep.com
10.35.16.1  virtual2.aucegep.com
```

```
10.35.16.1  virtual1.aucegep.com
10.35.16.1  virtual2.aucegep.com
```

Figure 18 Mapping custom domain names to the local server IP inside /etc/hosts.

These lines map the domain names to our Apache server's IP address.

Why do we do this on both machines?

- **On the AlmaLinux server**, this allows Apache to interpret ServerName directives and resolve virtual hostnames locally (useful for testing or log output).
- **On the Ubuntu client**, this allows us to test the websites in a browser using their domain names instead of typing IP addresses manually.

Test the name resolution

To confirm that the system now recognizes the domain names, I used the ping command from both machines:

```
root@server07 ~ $ ping -c 3 virtual1.acegep.com
PING virtual1.acegep.com (10.35.16.1) 56(84) bytes of data.
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=1 ttl=64 time=0.086 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=2 ttl=64 time=0.141 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=3 ttl=64 time=0.087 ms

--- virtual1.acegep.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2050ms
rtt min/avg/max/mdev = 0.086/0.104/0.141/0.025 ms
root@server07 ~ $
```

```
root@server07 ~ $ ping -c 3 virtual2.acegep.com
PING virtual2.acegep.com (10.35.16.1) 56(84) bytes of data.
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=1 ttl=64 time=0.044 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=2 ttl=64 time=0.132 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=3 ttl=64 time=0.153 ms

--- virtual2.acegep.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2085ms
rtt min/avg/max/mdev = 0.044/0.109/0.153/0.047 ms
root@server07 ~ $
```

```
gkeymole@client07:~$ ping -c 3 virtual1.acegep.com
PING virtual1.acegep.com (10.35.16.1) 56(84) bytes of data.
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=1 ttl=64 time=0.987 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=2 ttl=64 time=1.12 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=3 ttl=64 time=1.06 ms

--- virtual1.acegep.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 0.987/1.054/1.120/0.054 ms
gkeymole@client07:~$
```

```
gkeymole@client07:~$ ping -c 3 virtual2.acegep.com
PING virtual2.acegep.com (10.35.16.1) 56(84) bytes of data.
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=1 ttl=64 time=0.588 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=2 ttl=64 time=1.29 ms
64 bytes from virtual1.acegep.com (10.35.16.1): icmp_seq=3 ttl=64 time=0.950 ms

--- virtual2.acegep.com ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2039ms
rtt min/avg/max/mdev = 0.588/0.941/1.287/0.285 ms
gkeymole@client07:~$
```

Figure 19 Verifying that name resolution works correctly using ping.

Configure Apache Virtual Hosts in httpd.conf

In this step, I configured Apache to handle multiple websites running on the same server IP (10.35.16.1) but using different **TCP ports** and **domain names**. This technique is used in real-world web servers to host multiple applications or websites on a single machine.

Virtual Hosts: What Are They?

A **virtual host** is an Apache configuration block that defines how to serve different websites based on:

- The **IP address**
- The **domain name**
- The **port** used to connect

Since I'm reusing the same IP for all sites, the only way Apache can tell them apart is through the **domain + port combination**.

Edit the Apache Configuration File

I opened the main Apache config file:

```
root@server07 ~ $ vim /etc/httpd/conf/httpd.conf
```

Add Listen Directives

Before Apache can respond to connections on port 8000, we need to tell it to listen on that port. At the top of the file. These lines allow Apache to handle incoming traffic on both port 80 (default HTTP) and port 8000.

```
Listen 10.35.16.1:80
Listen 10.35.16.1:8000
```

Figure 20 Configuring Apache to listen on ports 80 and 8000.

Create Virtual Host Blocks

Below the DocumentRoot section of the file, I added four <VirtualHost> blocks. Each one is assigned a unique combination of domain + port, and points to a different folder created earlier in /var/www/virtuals.

Here's the full configuration I added:

```
# TASK 2 - Name + Port Virtual Hosts (1 IP, 4 ports)
<VirtualHost 10.35.16.1:80>
    ServerName virtual1.aucegep.com
    DocumentRoot /var/www/virtuals/virtual1_80
    ErrorLog /var/www/virtuals/virtual1_80/error.log
    CustomLog /var/www/virtuals/virtual1_80/access.log combined
</VirtualHost>

<VirtualHost 10.35.16.1:8000>
    ServerName virtual1.aucegep.com
    DocumentRoot /var/www/virtuals/virtual1_8000
    ErrorLog /var/www/virtuals/virtual1_8000/error.log
    CustomLog /var/www/virtuals/virtual1_8000/access.log combined
</VirtualHost>

<VirtualHost 10.35.16.1:80>
    ServerName virtual2.aucegep.com
    DocumentRoot /var/www/virtuals/virtual2_80
    ErrorLog /var/www/virtuals/virtual2_80/error.log
    CustomLog /var/www/virtuals/virtual2_80/access.log combined
</VirtualHost>

<VirtualHost 10.35.16.1:8000>
    ServerName virtual2.aucegep.com
    DocumentRoot /var/www/virtuals/virtual2_8000
    ErrorLog /var/www/virtuals/virtual2_8000/error.log
    CustomLog /var/www/virtuals/virtual2_8000/access.log combined
</VirtualHost>
```

Figure 21 Apache <VirtualHost> blocks configured for four combinations of domain and port

Each block does the following:

- **VirtualHost IP:port**: Tells Apache when to apply the rule (based on incoming request).
- **DocumentRoot**: Tells Apache where to find the files to serve.
- **ServerName**: Specifies the expected hostname to match.

How Apache uses this configuration: Apache will compare:

- The IP address
- The port
- The domain name

From the incoming request and match it with the corresponding <VirtualHost> block. That's how it knows which folder to serve for each case.

Create Separate Error and Access Log Subdirectories

Each virtual host block in Apache includes a logging section to track incoming requests and potential errors. While older configurations sometimes used the TransferLog directive for access logs, I chose to use CustomLog, which is now the recommended and more modern approach.

Inside each virtual host folder, I initially considered creating a dedicated logs/ subdirectory. However, Apache does not require this if I define the full path for the log files directly in the httpd.conf configuration. Instead, I placed each log file in its respective root directory:

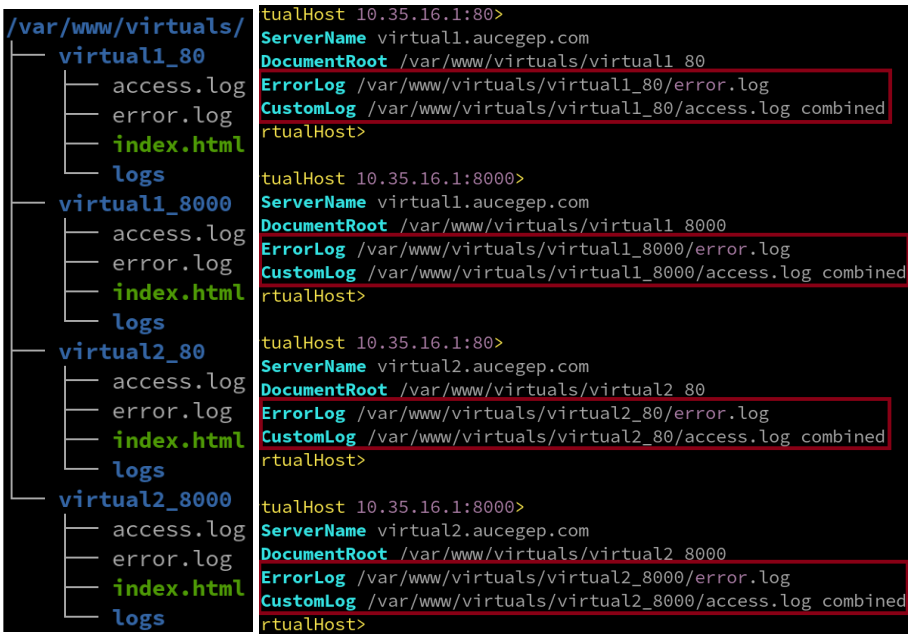

Figure 22 Apache virtual host block using CustomLog to define a detailed access log.

This ensures:

- Each virtual host logs its errors in a **separate file**.
- Each site logs access details with **full information** using the combined format.

What's the Difference Between TransferLog and CustomLog?

- TransferLog: Logs access data using the default common log format (limited information).
- CustomLog: Allows you to specify the format (e.g., common, combined, or custom). It is more flexible and modern.

Apache now favors CustomLog because it supports advanced log formats like combined, which includes important information such as the user agent (browser type) and referrer (previous page). This level of detail makes it easier to analyze traffic and troubleshoot issues.

If I had used TransferLog, I would not be able to customize the format easily, and the logs would have provided less insight.

Add Secondary IP to the Server

To isolate the virtual hosts from the main server content, I added the secondary IP address 10.35.16.1 to the AlmaLinux server.

```
root@server07 ~ $ nmcli connection modify LAN1 +ipv4.addresses 10.35.16.1/24
root@server07 ~ $ nmcli con down LAN1 ; nmcli con up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnec
tion/3)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/4)
root@server07 ~ $
```

Figure 23 Assigning secondary IP 10.35.16.1 to the Apache server interface.

Configure Ubuntu Client for Testing

To test the virtual hosts from a browser, I added another IP to the Ubuntu client:

```
gkeymole@client07:~$ sudo nmcli con mod LAN1 ipv4.addresses 10.35.16.2/24
gkeymole@client07:~$ sudo nmcli con down LAN1 ; sudo nmcli con up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/1)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/2)
```

Figure 24 Adding client IP 10.35.16.2 to ensure it can access the virtual hosts.

TASK 2: Validation: Port-Based Virtual Hosts

After completing the virtual host configurations, I tested each one to ensure that Apache correctly served the intended content based on the combination of hostname and port.

To do this, I opened the following URLs in the browser from my Ubuntu client:

- http://virtual1.aucegep.com
- http://virtual1.aucegep.com:8000
- http://virtual2.aucegep.com
- http://virtual2.aucegep.com:8000

Each virtual host loaded successfully and displayed a different message, confirming that Apache correctly routed requests based on hostname and port.

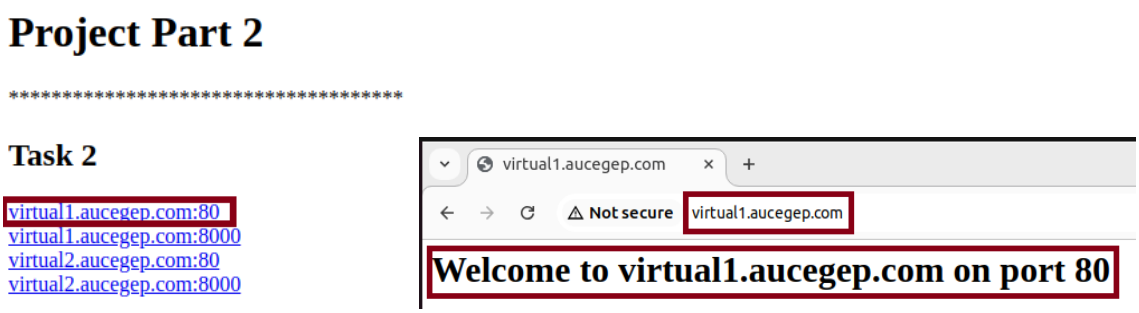

Figure 25 Homepage for virtual1.aucegep.com:80 loading from /var/www/virtuals/virtual1_80.

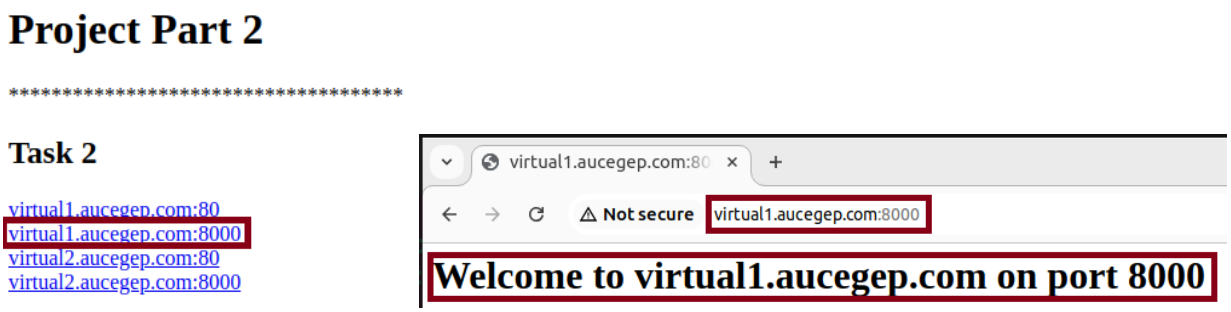

Figure 26 Homepage for virtual1.aucegep.com:8000 loading from /var/www/virtuals/virtual1_8000.

Figure 27 Homepage for virtual2.aucegep.com:80.

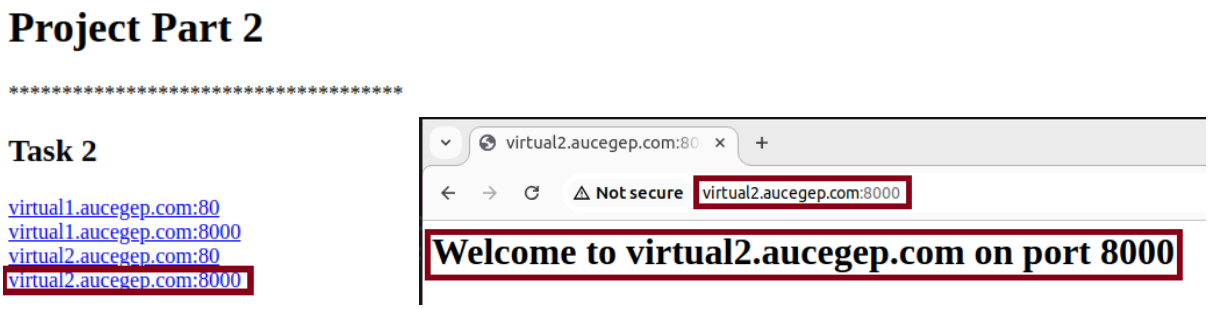

Figure 28 Homepage for virtual2.aucegep.com:8000.

TASK 3 – Name-Based Virtual Hosts with Access Control

In this task, I configured four virtual websites using **name-based hosting**. All of them share the same IP address (10.35.17.1), but each one uses a unique domain name and/or port. Two of these websites are accessible to everyone, while the other two are restricted to a specific subnet (10.35.17.0/24). This simulates how companies often restrict sensitive web portals, such as development or pre-production environments, to internal users only.

The four websites configured in this task are:

- **www.ici.com:80** → /var/www/virtuals/ici — Accessible to everyone
- **intranet.ici.com:80** → /var/www/virtuals/intranet — Accessible to everyone
- **intranet.ici.com:8000** → /var/www/virtuals/pre_production — Only accessible from subnet 10.35.17.0/24
- **development.ici.com:80** → /var/www/virtuals/development — Only accessible from subnet 10.35.17.0/24

Add the Secondary IP to the Apache Server

First, I added a new IP address (10.35.17.1) to the AlmaLinux server's network interface so that Apache could use it for this task's virtual hosts.

```
root@server07 ~ $ nmcli connection modify LAN1 +ipv4.addresses 10.35.17.1/24
root@server07 ~ $ nmcli con down LAN1 ; nmcli con up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnec
tion/4)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/5)
root@server07 ~ $ ip a | grep 10.35.17.1
    inet 10.35.17.1/24 brd 10.35.17.255 scope global noprefixroute ens192
root@server07 ~ $
```

Figure 29 Assigning the IP address 10.35.17.1 to the server.

Update the Ubuntu Client's IP for Testing

To test the subnet restriction later, I assigned another IP to the Ubuntu client from a different subnet (10.35.16.2). This way, I could confirm that access to restricted websites is denied outside the allowed subnet.

```
gkeymole@client07:~$ nmcli connection modify LAN1 +ipv4.addresses "10.35.17.2/24"
gkeymole@client07:~$ nmcli con down LAN1 ; nmcli con up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/2)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/3)
gkeymole@client07:~$ ip a | grep 10.35.17.2
    inet 10.35.17.2/24 brd 10.35.17.255 scope global noprefixroute ens33
gkeymole@client07:~$
```

Figure 30 Adding an IP from 10.35.16.0/24 to simulate access denial for restricted hosts.

Update /etc/hosts on Server and Client

Since these virtual hosts use domain names, and we are not using a real DNS server in this lab, I manually mapped the domain names to the server IP in the /etc/hosts file on **both the AlmaLinux server and the Ubuntu client**.

```
root@server07 ~ $ echo "10.35.17.1 www.ici.com intranet.ici.com development.ici.com" >> /etc/hosts
root@server07 ~ $ cat /etc/hosts
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6
10.35.16.1 virtual1.aucegep.com
10.35.16.1 virtual2.aucegep.com
10.35.17.1 www.ici.com intranet.ici.com development.ici.com
root@server07 ~ $
```

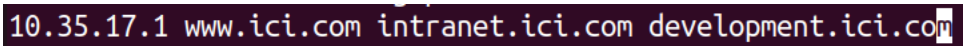

Figure 31 Mapping hostnames to 10.35.17.1 in /etc/hosts.

Create the Website Directories and Homepages

I created a directory for each of the four virtual hosts and added a simple index.html file to each one so I could visually confirm which site was being served.

```
root@server07 ~ $ mkdir -p /var/www/virtuals/ici /var/www/virtuals/intranet /var/www/virtuals/pre_productio
n /var/www/virtuals/development
root@server07 ~ $
```

```
root@server07 ~ $ echo "<h1>Welcome to www.ici.com</h1>" > /var/www/virtuals/ici/index.html
root@server07 ~ $ echo "<h1>Welcome to intranet.ici.com on port 80</h1>" > /var/www/virtuals/intranet/index
.html
root@server07 ~ $ echo "<h1>Welcome to intranet.ici.com on port 8000</h1>" > /var/www/virtuals/pre_producti
on/index.html
root@server07 ~ $ echo "<h1>Welcome to development.ici.com</h1>" > /var/www/virtuals/development/index.html
root@server07 ~ $
```

Figure 32 Creating directories and HTML pages for each virtual host.

Configure Apache Virtual Host Blocks in httpd.conf

Next, I edited /etc/httpd/conf/httpd.conf and added the following <VirtualHost> blocks. These blocks define the domain, port, document root, and access rules.

```
# --- TASK 3 ---
<VirtualHost 10.35.17.1:80>
    ServerName www.ici.com
    DocumentRoot /var/www/virtuals/ici
    ErrorLog /var/www/virtuals/ici/error.log
    CustomLog /var/www/virtuals/ici/access.log combined
</VirtualHost>

<VirtualHost 10.35.17.1:80>
    ServerName intranet.ici.com
    DocumentRoot /var/www/virtuals/intranet
    ErrorLog /var/www/virtuals/intranet/error.log
    CustomLog /var/www/virtuals/intranet/access.log combined
</VirtualHost>

<VirtualHost 10.35.17.1:8000>
    ServerName intranet.ici.com
    DocumentRoot /var/www/virtuals/pre_production
    <Directory "/var/www/virtuals/pre_production">
        Require ip 10.35.17.0/24
    </Directory>
    ErrorLog /var/www/virtuals/pre_production/error.log
    CustomLog /var/www/virtuals/pre_production/access.log combined
</VirtualHost>

<VirtualHost 10.35.17.1:80>
    ServerName development.ici.com
    DocumentRoot /var/www/virtuals/development
    <Directory "/var/www/virtuals/development">
        Require ip 10.35.17.0/24
    </Directory>
    ErrorLog /var/www/virtuals/development/error.log
    CustomLog /var/www/virtuals/development/access.log combined
</VirtualHost>
```

Figure 33 Apache virtual host blocks for Task 3, including subnet restrictions.

Add Apache Listen Directives for Task 3

Since some of the virtual hosts in this task use both **port 80** and **port 8000**, I added the following Listen directives in httpd.conf. I also included a comment to clearly indicate these lines are related to Task 3.

```
#TASK3
Listen 10.35.17.1:80
Listen 10.35.17.1:8000
```

Figure 34 Apache is instructed to listen on the IP and ports required by Task 3, clearly labeled for readability.

TASK 3: Validation – Name-Based Virtual Hosts with Subnet-Based Access Control

After configuring the four virtual hosts and setting up the proper subnet-based restrictions, I performed validation using both the browser and the curl command-line tool. These tests confirmed that Apache serves or restricts access based on the source IP address and requested domain name, as required.

Access to www.ici.com (Port 80)

Using the Ubuntu client with IP **10.35.17.2**, I opened the following link in the browser. The homepage loaded successfully, indicating that there are no restrictions for this virtual host.

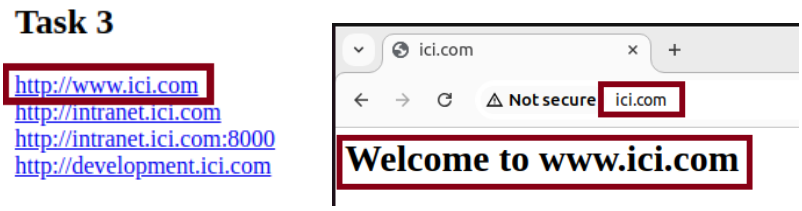

Figure 35 Homepage for www.ici.com loaded successfully on port 80 with no access restrictions.

Access to intranet.ici.com (Port 80)

From the same client, I tested the second host. This page also loaded without issue, confirming unrestricted access as expected.

Figure 36 Homepage for intranet.ici.com loaded successfully on port 80, accessible to everyone.

Restricted Access to intranet.ici.com:8000

To verify subnet-based restriction, I simulated access from the **unauthorized subnet (10.35.16.0/24)** by using the curl command with the --interface flag. Apache returned a **403 Forbidden** response, confirming that access to the **pre_production** directory is restricted outside the **10.35.17.0/24** subnet.

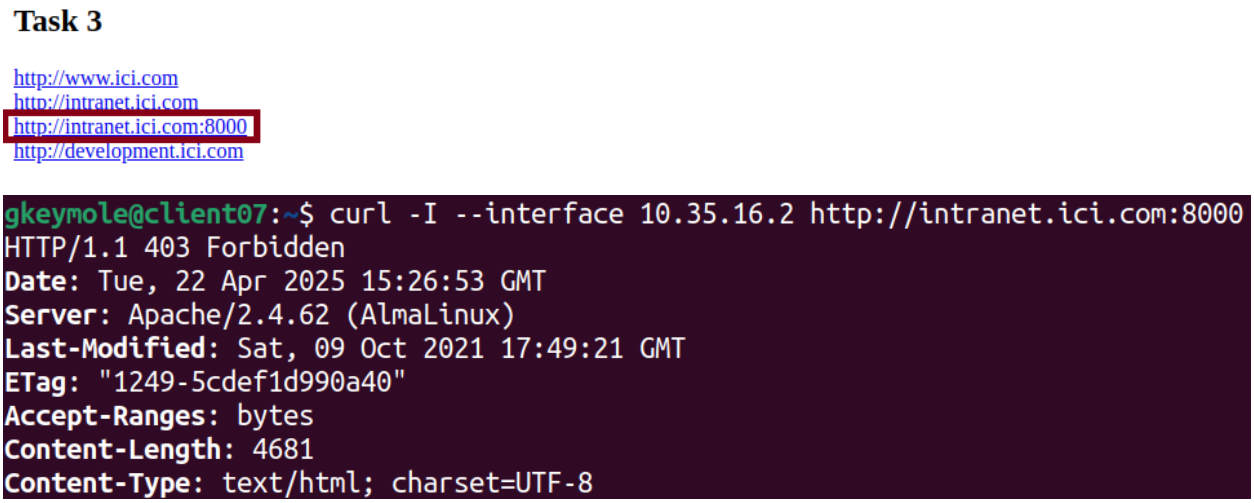

Figure 37 Attempting to access intranet.ici.com:8000 from 10.35.16.2 returns a 403 Forbidden error, validating the subnet restriction for pre_production.

Restricted Access to development.ici.com (Port 80)

Next, I tested the fourth virtual host, also protected by subnet restriction. Again, the result was a **403 Forbidden**, which indicates that the **development** directory is protected and only accessible from the **10.35.17.0/24** subnet.

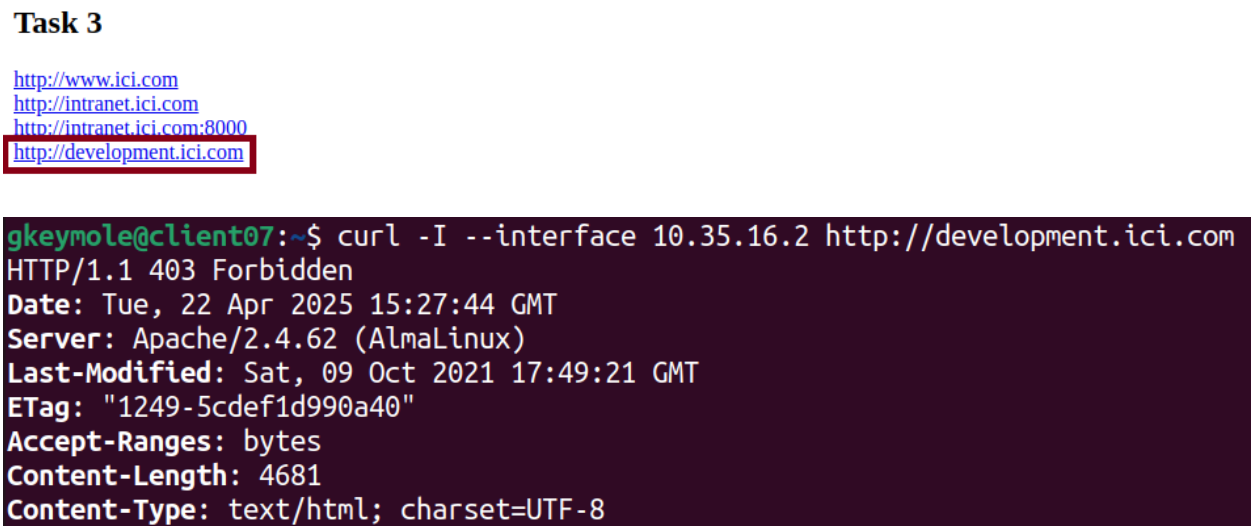

Figure 38 Access to development.ici.com from 10.35.16.2 is denied, confirming the correct subnet-based access control.

TASK 4 – Virtual Web Servers by IP and Port

In this task, we configure **four Apache virtual hosts**, each identified by a **unique port number** but sharing the same IP address (10.35.16.1). This method simulates a real-world scenario where different applications are hosted on different ports of the same server.

Each virtual host is assigned a unique **DocumentRoot directory** and a dedicated **log file**, following good web server administration practices.

Create the directory structure

We create a **separate directory** for each web server under /var/www/virtuals/q4. This helps us stay organized and ensures each virtual host has its own isolated content.

```
root@server07 ~ $ mkdir -p /var/www/virtuals/q4/sales /var/www/virtuals/q4/admin /var/www/virtuals/q4/thing
 /var/www/virtuals/q4/other
root@server07 ~ $
```

Figure 39 Creating directories for each virtual server under /var/www/virtuals/q4/.

Create index.html for each site

Inside each directory, we create a simple index.html to confirm that each virtual host displays the correct content.

```
root@server07 ~ $ echo "<h1>Welcome to the SALES server on port 8080</h1>" > /var/www/virtuals/q4/sales/ind
ex.html
root@server07 ~ $ echo "<h1>Welcome to the ADMIN server on port 8081</h1>" > /var/www/virtuals/q4/admin/ind
ex.html
root@server07 ~ $ echo "<h1>Welcome to the THING server on port 8082</h1>" > /var/www/virtuals/q4/thing/ind
ex.html
root@server07 ~ $ echo "<h1>Welcome to the OTHER server on port 8083</h1>" > /var/www/virtuals/q4/other/ind
ex.html
root@server07 ~ $
```

Figure 40 Adding HTML content to distinguish each server in the browser.

Update Apache httpd.conf to listen on required ports

Apache must be explicitly told to listen on each of the custom ports:

```
#TASK 4
Listen 10.35.16.1:8080
Listen 10.35.16.1:8081
Listen 10.35.16.1:8082
Listen 10.35.16.1:8083
```

Figure 41 Updating httpd.conf to ensure Apache listens on ports 8080–8083.

Define the VirtualHost blocks in httpd.conf

We now add a <VirtualHost> block for each of the four web servers:

```
# --- TASK 4 ---
<VirtualHost 10.35.16.1:8080>
    ServerName sales.q4.local
    DocumentRoot /var/www/virtuals/q4/sales
    ErrorLog /var/www/virtuals/q4/sales/error.log
    CustomLog /var/www/virtuals/q4/sales/access.log combined
</VirtualHost>

<VirtualHost 10.35.16.1:8081>
    ServerName admin.q4.local
    DocumentRoot /var/www/virtuals/q4/admin
    ErrorLog /var/www/virtuals/q4/admin/error.log
    CustomLog /var/www/virtuals/q4/admin/access.log combined
</VirtualHost>

<VirtualHost 10.35.16.1:8082>
    ServerName thing.q4.local
    DocumentRoot /var/www/virtuals/q4/thing
    ErrorLog /var/www/virtuals/q4/thing/error.log
    CustomLog /var/www/virtuals/q4/thing/access.log combined
</VirtualHost>

<VirtualHost 10.35.16.1:8083>
    ServerName other.q4.local
    DocumentRoot /var/www/virtuals/q4/other
    ErrorLog /var/www/virtuals/q4/other/error.log
    CustomLog /var/www/virtuals/q4/other/access.log combined
</VirtualHost>
```

Figure 42 Apache VirtualHost blocks for port-based web servers.

Each block defines:

- The **port and IP** combination.
- The directory where the HTML files are stored.
- Separate **error** and **access** logs for each server.

TASK 4: Validation – Virtual Web Servers by IP and Port

After configuring the VirtualHost blocks and directories, we validated that each virtual web server is accessible using the correct IP address and port. The Apache server responds with the correct web page content based on the requested port.

Validation 1 – Accessing Sales Server on Port 8080

From the Ubuntu client, we opened the following URL in a web browser:

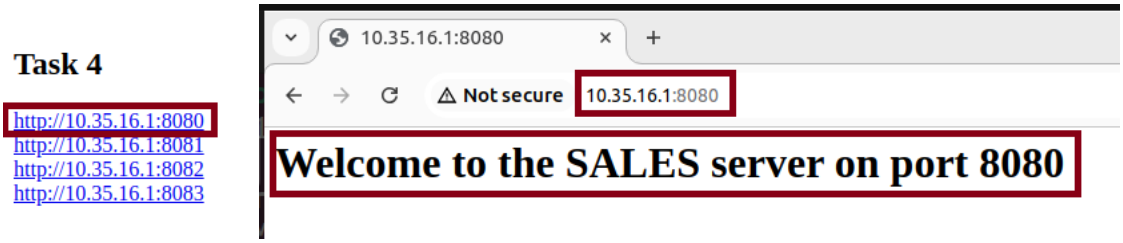

Figure 43 Homepage of the Sales server displayed successfully from /var/www/virtuals/q4/sales.

Validation 2 – Accessing Admin Server on Port 8081

From the Ubuntu client, we opened the following URL:

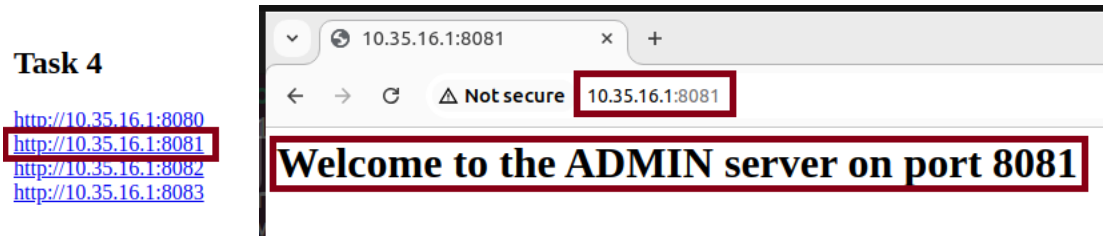

Figure 44 Homepage of the Admin server displayed successfully from /var/www/virtuals/q4/admin.

Validation 3 – Accessing Thing Server on Port 8082

Browser URL:

Figure 45 Homepage of the Thing server displayed successfully from /var/www/virtuals/q4/thing.

Validation 4 – Accessing Other Server on Port 8083

Browser URL:

Figure 46 Homepage of the Other server displayed successfully from /var/www/virtuals/q4/other.

TASK 5 – DYNAMIC VIRTUAL HOSTING

In this final task, we set up multiple virtual websites that all use the **same IP address (10.50.1.1)** and **same port (80)**. The difference between them is their **domain names**, and we use **Apache's VirtualDocumentRoot directive** to serve them from different folders automatically. This method is called **dynamic virtual hosting**, and it's very useful when managing lots of websites efficiently without writing a <VirtualHost> block for each one.

We configured the following five virtual websites:

1. **www.itmt.com - /var/www/virtuals/q5/com/itmt/www/**
2. **www.itmt.ca - /var/www/virtuals/q5/ca/itmt/www/**
3. **www2.itmt.com - /var/www/virtuals/q5/com/itmt/www2/**
4. **www.montmo.com - /var/www/virtuals/q5/com/montmo/www/**
5. **www.montmo.ca - /var/www/virtuals/q5/ca/montmo/www/**

Create Directory Structure

We first created the necessary directory with the mkdir -p command to create all required folders in one shot:

```
root@server07 ~ $ tree /var/www/virtuals/q5
/var/www/virtuals/q5
├── ca
│   ├── itmt
│   │   └── www
│   │       └── index.html
│   └── montmo
│       └── www
│           └── index.html
├── com
│   ├── itmt
│   │   ├── www
│   │   │   └── index.html
│   │   └── www2
│   │       └── index.html
│   └── montmo
│       └── www
│           └── index.html
└── logs
    └── virtual_error_q5_log
```

```
root@server07 ~ $ mkdir -p /var/www/virtuals/q5/com/itmt/www /var/www/virtuals/q5/ca/itmt/www /var/www/virtuals/q5/com/itmt/www2 /var/www/virtuals/q5/com/montmo/www /var/www/virtuals/q5/ca/montmo/www
root@server07 ~ $
```

Figure 47 Directory structure created for dynamic virtual hosting under /var/www/virtuals/q5.

Add a Homepage for Each Website

Inside each folder, we created a simple index.html page with a custom message to identify the site. We repeated this for all 5 sites, adjusting the message for each one.

```
root@server07 ~ $ echo "<h1>Welcome to www.itmt.com</h1>" > /var/www/virtuals/q5/com/itmt/www/index.html
root@server07 ~ $ echo "<h1>Welcome to www.itmt.ca</h1>" > /var/www/virtuals/q5/ca/itmt/www/index.html
root@server07 ~ $ echo "<h1>Welcome to www2.itmt.com</h1>" > /var/www/virtuals/q5/com/itmt/www2/index.html
root@server07 ~ $ echo "<h1>Welcome to www.montmo.com</h1>" > /var/www/virtuals/q5/com/montmo/www/index.htm
l
root@server07 ~ $ echo "<h1>Welcome to www.montmo.ca</h1>" > /var/www/virtuals/q5/ca/montmo/www/index.html
root@server07 ~ $
```

Figure 48 Custom homepage placed in each dynamic virtual host directory.

Update the Apache Configuration

We opened the main Apache configuration file. At the top, we added a Listen directive to ensure Apache accepts traffic on the new IP:

```
#TASK 5
Listen 10.50.1.1:80
```

Figure 49 Listen 10.50.1.1:80 added to support dynamic hosting IP.

Then we added the following dynamic <VirtualHost> block:

```
# TASK 5 - Dynamic Virtual Hosting
<VirtualHost 10.50.1.1:80>
    ServerAdmin webmaster@localhost
    UseCanonicalName Off
    VirtualDocumentRoot "/var/www/virtuals/q5/%3/%2/%1"
    ErrorLog /var/www/virtuals/q5/logs/virtual_error_q5_log
</VirtualHost>
```

Figure 50 Dynamic virtual hosting configuration using VirtualDocumentRoot.

Explanation:

VirtualDocumentRoot tells Apache where to find the site's folder based on the **TLD**, **domain**, and **prefix**.

The placeholders:

- %1 → prefix (e.g., www)
- %2 → domain name (e.g., itmt)
- %3 → TLD (e.g., com)

This matches the structure we created earlier.

Create Log Directory

To avoid startup errors, we created the log folder specified in the ErrorLog directive:

```
root@server07 ~ $ mkdir -p /var/www/virtuals/q5/logs
root@server07 ~ $
```

Figure 51 Log folder created to store the dynamic virtual host error logs.

Update /etc/hosts File (Server + Client)

Since these are fake domain names and not real DNS entries, we mapped them locally.

On both the **AlmaLinux server** and **Ubuntu client**, we added:

```
10.50.1.1   www.itmt.com www.itmt.ca www2.itmt.com www.montmo.com www.montmo.ca
```

```
10.50.1.1   www.itmt.com www.itmt.ca www2.itmt.com www.montmo.com www.montmo.ca
```

Figure 52 Mapping dynamic virtual hostnames to IP 10.50.1.1 in /etc/hosts.

Assign the IP Address to the Server

We added the IP address 10.50.1.1 to our server:

```
ens192: connected to LAN1
        "VMware VMXNET3"
        ethernet (vmxnet3), 00:0C:29:78:C7:EE, hw, mtu 1500
        inet4 10.50.1.1/24
        inet4 10.35.17.1/24
        inet4 10.35.16.1/24
        inet4 192.168.50.10/24
        route4 192.168.50.0/24 metric 101
        route4 10.35.16.0/24 metric 101
        route4 10.35.17.0/24 metric 101
        route4 10.50.1.0/24 metric 101
        inet6 fe80::7782:9465:7774:23df/64
        route6 fe80::/64 metric 1024
```

Figure 53 IP address 10.50.1.1 assigned to the Apache server interface.

Add IP from 10.50.1.0/24 Subnet to Ubuntu Client

To allow the **Ubuntu client** to access the virtual websites hosted on 10.50.1.1, we assigned an IP from the same subnet (e.g., 10.50.1.2) to its network interface.

```
gkeymole@client07:~$ nmcli connection modify LAN1 +ipv4.addresses "10.50.1.2/24"
gkeymole@client07:~$ nmcli connection down LAN1 ; nmcli connection up LAN1
Connection 'LAN1' successfully deactivated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/3)
Connection successfully activated (D-Bus active path: /org/freedesktop/NetworkManager/ActiveConnection/4)
```

Figure 54 Adding secondary IP 10.50.1.2 to the Ubuntu client to enable communication with the dynamic virtual host server.

Check if Dynamic Virtual Host Module is Enabled

Before configuring **dynamic virtual hosts**, we must confirm that Apache's **vhost_alias module** is loaded. This module allows the use of the VirtualDocumentRoot directive, which is required for mapping incoming requests to dynamically structured folders.

To check if the module is active, I ran:

```
root@server07 ~ $ httpd -M | grep vhost_alias
 vhost_alias_module (shared)
root@server07 ~ $
```

Figure 55 Verifying that the vhost_alias_module is loaded, confirming support for dynamic virtual

If it was not enabled, I would have uncommented or added this line in the Apache configuration:

LoadModule vhost_alias_module modules/mod_vhost_alias.so

Note. Dynamic virtual hosts let us simplify configuration by using one <VirtualHost> block and a VirtualDocumentRoot directive instead of repeating <VirtualHost> for each site.

TASK 5: Validation – Dynamic Virtual Hosting

After configuring dynamic virtual hosting using the VirtualDocumentRoot directive, I tested access to each of the five virtual servers to confirm Apache was correctly serving each site from its respective directory.

To do this, I simply opened the following URLs in a web browser from the Ubuntu client:

- **www.itmt.com - /var/www/virtuals/q5/com/itmt/www/**
- **www.itmt.ca - /var/www/virtuals/q5/ca/itmt/www/**
- **www2.itmt.com - /var/www/virtuals/q5/com/itmt/www2/**
- **www.montmo.com - /var/www/virtuals/q5/com/montmo/www/**
- **www.montmo.ca - /var/www/virtuals/q5/ca/montmo/www/**

Each website successfully loaded its corresponding homepage content, which had been defined in the proper folder structure based on TLD, company name, and prefix.

www.itmt.com

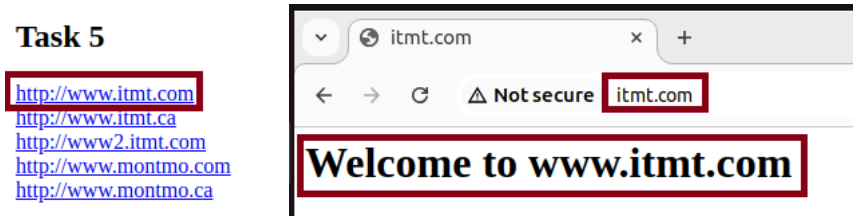

Figure 56 Homepage for www.itmt.com displayed from /var/www/virtuals/q5/com/itmt/www.

www.itmt.ca

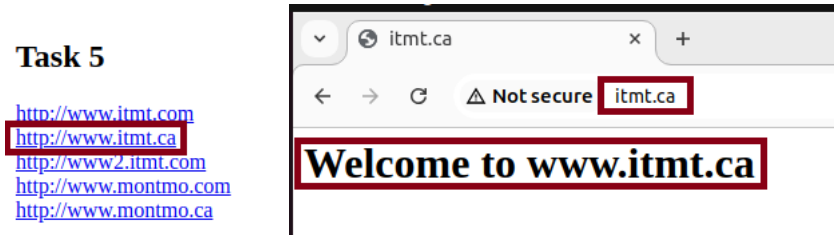

Figure 57 Homepage for www.itmt.ca displayed from /var/www/virtuals/q5/ca/itmt/www.

www2.itmt.com

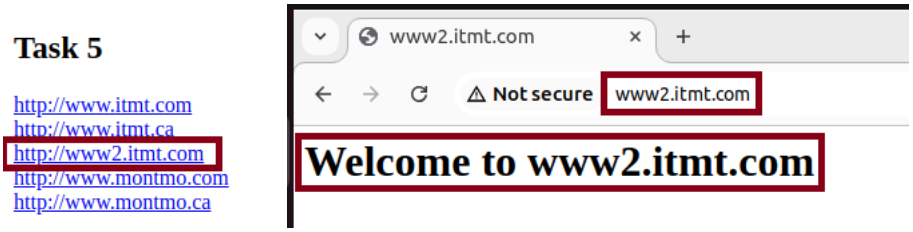

Figure 58 Homepage for www2.itmt.com displayed from /var/www/virtuals/q5/com/itmt/www2.

www.montmo.com

Figure 59 Homepage for www.montmo.com displayed from /var/www/virtuals/q5/com/montmo/www.

www.montmo.ca

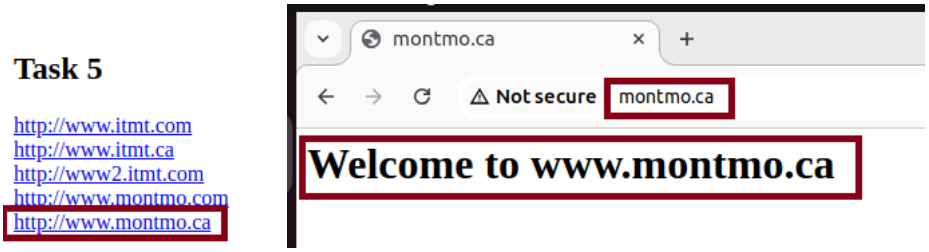

Figure 60 Homepage for www.montmo.ca displayed from /var/www/virtuals/q5/ca/montmo/www.

Additional Validation: Domain-to-IP Resolution with ping

To ensure the Ubuntu client was resolving all domain names correctly to the server IP 10.50.1.1, I ran the ping command for each site:

```
gkeymole@client07:~$ ping -c 2 www.itmt.com
PING www.itmt.com (10.50.1.1) 56(84) bytes of data.
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=1 ttl=64 time=0.707 ms
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=2 ttl=64 time=0.230 ms

--- www.itmt.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1026ms
rtt min/avg/max/mdev = 0.230/0.468/0.707/0.238 ms
gkeymole@client07:~$ ping -c 2 www.itmt.ca
PING www.itmt.com (10.50.1.1) 56(84) bytes of data.
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=1 ttl=64 time=0.530 ms
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=2 ttl=64 time=0.924 ms

--- www.itmt.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1043ms
rtt min/avg/max/mdev = 0.530/0.727/0.924/0.197 ms
gkeymole@client07:~$ ping -c 2 www2.itmt.com
PING www.itmt.com (10.50.1.1) 56(84) bytes of data.
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=1 ttl=64 time=0.395 ms
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=2 ttl=64 time=0.476 ms

--- www.itmt.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1047ms
rtt min/avg/max/mdev = 0.395/0.435/0.476/0.040 ms
gkeymole@client07:~$ ping -c 2 www.montmo.com
PING www.itmt.com (10.50.1.1) 56(84) bytes of data.
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=1 ttl=64 time=0.450 ms
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=2 ttl=64 time=0.261 ms

--- www.itmt.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1015ms
rtt min/avg/max/mdev = 0.261/0.355/0.450/0.094 ms
gkeymole@client07:~$ ping -c 2 www.montmo.ca
PING www.itmt.com (10.50.1.1) 56(84) bytes of data.
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=1 ttl=64 time=0.361 ms
64 bytes from www.itmt.com (10.50.1.1): icmp_seq=2 ttl=64 time=0.236 ms

--- www.itmt.com ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1013ms
rtt min/avg/max/mdev = 0.236/0.298/0.361/0.062 ms
gkeymole@client07:~$
```

Figure 61 Successful ping responses for all five domain names showing that name resolution to 10.50.1.1 is working on the Ubuntu client.

Validation: curl Command

For extra verification, I also used the curl command from the Ubuntu client to fetch the homepage of each domain and confirm Apache's response headers:

```
gkeymole@client07:~$ curl -I http://www.itmt.com
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2025 23:50:05 GMT
Server: Apache/2.4.62 (AlmaLinux)
Last-Modified: Tue, 22 Apr 2025 22:34:15 GMT
ETag: "21-6336595a964be"
Accept-Ranges: bytes
Content-Length: 33
Content-Type: text/html; charset=UTF-8

gkeymole@client07:~$ curl -I http://www.itmt.ca
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2025 23:50:12 GMT
Server: Apache/2.4.62 (AlmaLinux)
Last-Modified: Tue, 22 Apr 2025 22:34:33 GMT
ETag: "20-6336596b689e2"
Accept-Ranges: bytes
Content-Length: 32
Content-Type: text/html; charset=UTF-8

gkeymole@client07:~$ curl -I http://www2.itmt.com
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2025 23:50:18 GMT
Server: Apache/2.4.62 (AlmaLinux)
Last-Modified: Tue, 22 Apr 2025 22:34:58 GMT
ETag: "22-633659836d312"
Accept-Ranges: bytes
Content-Length: 34
Content-Type: text/html; charset=UTF-8

gkeymole@client07:~$ curl -I http://www.montmo.com
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2025 23:50:25 GMT
Server: Apache/2.4.62 (AlmaLinux)
Last-Modified: Tue, 22 Apr 2025 22:35:19 GMT
ETag: "23-633659972dbe9"
Accept-Ranges: bytes
Content-Length: 35
Content-Type: text/html; charset=UTF-8

gkeymole@client07:~$ curl -I http://www.montmo.ca
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2025 23:50:44 GMT
Server: Apache/2.4.62 (AlmaLinux)
Last-Modified: Tue, 22 Apr 2025 22:35:29 GMT
ETag: "22-633659a0c4892"
Accept-Ranges: bytes
Content-Length: 34
Content-Type: text/html; charset=UTF-8
```

Figure 62 curl -I output showing HTTP/1.1 200 OK responses from all five dynamic virtual hosts.

Conclusion

This project provided a comprehensive, hands-on opportunity to explore the full range of Apache virtual hosting techniques on a Linux server. Over the course of five detailed tasks, I successfully configured and tested:

- **Name-based virtual hosts** using different domains.
- **Port-based virtual hosts** to separate content on the same IP address.
- **Access-controlled virtual hosts** restricted by subnet.
- **Virtual hosts separated by IP and port** for hosting distinct services.
- **Dynamic virtual hosting** using the powerful VirtualDocumentRoot directive.

Through these exercises, I gained a deep understanding of how Apache interprets requests based on IP address, port, and domain name. I also became comfortable managing server configurations using the httpd.conf file, assigning correct permissions, handling SELinux contexts, and troubleshooting issues through validation techniques like curl, ping, and httpd -t.

Overall, this project reinforced key web server administration skills and helped me simulate real-world scenarios where a single Apache server can securely and efficiently host multiple websites for different clients or departments. The experience also improved my confidence in working with Linux-based server environments and will be valuable for future roles in system or network administration.